

'constexpr' feature macro concerns

I. Quick Introduction

In [P1032R1 "Misc constexpr bits"](#) a feature testing macro `__cpp_lib_constexpr_misc` was proposed.

That's not the best name for the macro. The name is not very informative. `__cpp_lib_constexpr_misc` also does not suite the needs of future constexprifications.

This paper discuss different approaches for constexpr feature testing macro.

II. Approaches

A. Single constexpr feature testing macro

Make a `__cpp_lib_constexpr` macro and increment its version on each new constexpr addition to the library.

Pros:

- Simple to use and to remember

Cons:

- Not vendor friendly - if there's a constexpr addition *A* followed by a constexpr addition *B* and *A* is hard to implement, then the vendor can not say that *B* is supported, even if it is implemented

B. Fuse constexpr feature testing into the feature testing macro of a component

Update the value of feature testing macro of a particular component. For example, update `__cpp_lib_string_view` with value 201811L because [P1032R1 "Misc constexpr bits"](#) touched the `string_view`.

Pros:

- Better granularity, more vendor friendly
- Simple to find

Cons:

- Not many Standard library classes have their own feture testing macro. So with this approach each constexpr addition adds a bunch of feature testing macro to the `[support.limits.general]`.

C. Have a feature testing macro per paper/proposal

This approach was taken by the [P1032R1](#) with its `__cpp_lib_constexpr_misc` macro.

Pros:

- Simple to detect that the particular paper is implemented by vendor

Cons:

- Not vendor friendly - paper could be very big and the implementor may wish to implement part of it right now, leaving the remaining parts for the next release.

III. Conclusions

Each approach has its own advantages and disadvantages. I'd like recommend the following generic approach:

- Each new constexpr paper should have either a new feature testing macro or should update the existing one.
- Introduce a `__cpp_lib_constexpr` macro and increase it on each LWG constexpr paper adoption.
- Listen to the Standard Library developers and users. Add feature testing macro on their request.

IV. LWG discussion

Discussion of the paper in Kona lead to the following conclusion: rename `__cpp_lib_constexpr_misc` to `__cpp_lib_constexpr` and bump the value on every change.

Note that for a more generic feature testing macro we should include it in every header or just not include it in some subset of headers. Otherwise it would be surprising for users. For example if we increment the macro value for some constexpr additions in `<vector>` the macro will not be available via `<vector>` inclusion.

V. Wording (relative to N4800)

Apply the following changes to the Table 36 "Standard library feature-test macros" in [support.limits.general]:

<code>__cpp_lib_constexpr</code>	<code>misc</code>	201811L	<code><array></code> <code><functional></code> <code><iterator></code> <code><string_view></code> <code><tuple></code> <code><utility></code> Any C++ library header from Table 19 or any C++ header for C library facilities from Table 20
----------------------------------	------------------------------	---------	---